Il processo software

Un **processo software** è la sequenza di attività necessarie per sviluppare un sistema software.

Attività principali di un processo software

Indipendentemente dal modello specifico utilizzato, tutti i processi software comprendono le seguenti fasi:

1. **Specifica**
    - Definizione di **cosa deve essere fatto**.
    - Obiettivo: chiarire i requisiti del sistema e le aspettative degli utenti.
2. **Progettazione e implementazione**
    - Ideazione dell'architettura del sistema e sviluppo del codice.
    - Obiettivo: tradurre i requisiti in un software funzionante.
3. **Validazione**
    - Verifica che il sistema **faccia quanto richiesto**.
    - Include test, revisione e controllo della qualità.
4. **Evoluzione**
    - Modifica o aggiornamento del sistema in base a nuove esigenze o correzione di problemi.
    - Obiettivo: mantenere il software utile e aggiornato nel tempo.

Modellare processi software

Un **modello di processo software** fornisce una rappresentazione astratta del processo stesso, permettendo di comprendere e gestire lo sviluppo del software in modo strutturato.

Caratteristiche principali

- **Suddivisione in attività**: definisce **cosa fare**, **quando farlo**, **chi è responsabile** e **quali risultati ottenere**.
- **Organizzazione delle attività**: stabilisce **l'ordine delle attività**, i criteri per **terminare un'attività** e passare alla successiva.
- **Esempio standard**: ISO 12207.

Esempio non software: preparazione di un dolce da forno

Un modello di sviluppo generico potrebbe essere:

1. Fare la spesa.
2. Impastare e mettere l'impasto nella teglia mentre il forno si scalda.
3. Infornare.
4. Quando pronto, rimuovere dal forno.

Modelli di ciclo di vita del software

I **modelli di ciclo di vita** descrivono come organizzare le attività di sviluppo software nel tempo.

1. Build-and-Fix (un «non-modello»)

- Approccio semplice, spesso caotico: si costruisce il software e si correggono i difetti man mano che emergono.
- Non segue una struttura formale, per questo è definito un «non-modello».

2. Modelli prescrittivi

Strutturano rigidamente le fasi del processo software:

- **Cascata (Waterfall)**: sviluppo sequenziale con fasi ben definite.
- **Modello a V**: simile alla cascata, ma con enfasi sulla **verifica e validazione** in parallelo.

- **Rapid Prototyping**: costruzione rapida di prototipi per chiarire i requisiti.
- **Modello incrementale**: il software viene sviluppato in **versioni parziali** (incrementi) fino al completamento.
- **Modello a spirale**: sviluppo iterativo con analisi del rischio ad ogni ciclo.

3. Unified Process

- Modello iterativo e **guidato dai casi d'uso**, che combina elementi prescrittivi e iterativi.

4. Modelli agili

- Approcci **flessibili e iterativi**, con focus su collaborazione e consegne frequenti:
  - **Extreme Programming (XP)**: enfatizza programmazione in coppia, test continui e feedback rapido.
  - **Scrum**: organizzazione del lavoro in **sprint** con ruoli e cerimonie specifiche.
  - **Continuous Delivery**: sviluppo e rilascio continuo di software in produzione.

Build-and-Fix

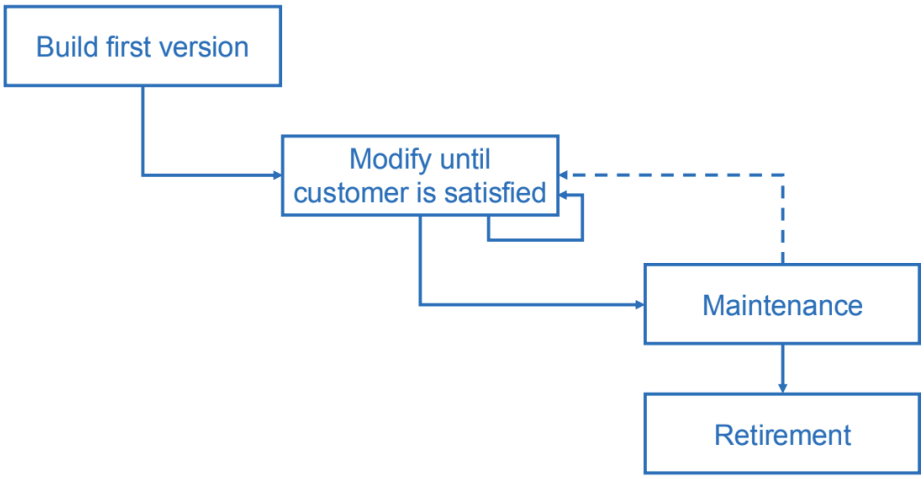
Il **Build-and-Fix** è un approccio informale allo sviluppo software, spesso definito un **«non-modello»**, perché manca di specifiche e progettazione strutturata.

Caratteristiche principali

- **Sviluppo senza specifica**: il sistema software viene realizzato **senza una chiara definizione dei requisiti**.
- **Sviluppo senza progettazione**: non viene pianificata l'architettura o l'organizzazione del codice.

- **Ciclo di modifica continuo**: lo sviluppatore scrive il programma e lo **modifica ripetutamente** fino a quando soddisfa le richieste del committente.

> Nota: questo approccio può funzionare per progetti molto piccoli o sperimentali, ma diventa insostenibile per sistemi complessi.



Quando usare il Build-and-Fix

Contesti possibili

- Può essere **adeguato solo per progetti molto piccoli**, ad esempio di circa 100 righe di codice.
- **Inappropriato per sistemi di dimensioni medio-grandi**.

Limiti principali

- La **manutenzione** senza specifica o documentazione progettuale è **molto difficile e costosa**.
- L'approccio può portare a **problemi imprevisti** durante lo sviluppo o dopo il rilascio.

Raccomandazione

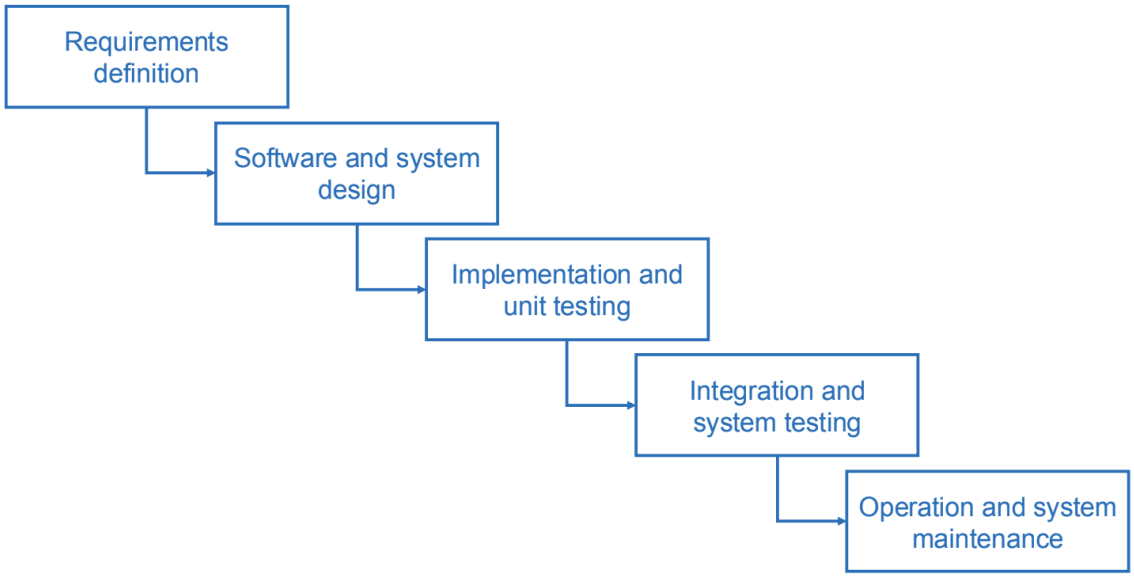
- È **preferibile adottare un vero modello di ciclo di vita** fin dall'inizio dello sviluppo, per ridurre rischi, costi e sorprese indesiderate.

Modello a Cascata

Il **modello a cascata** è il primo modello a distinguere chiaramente le fasi di un processo software, enfatizzando l'analisi e la progettazione **prima della codifica**.

Caratteristiche principali

- **Fasi sequenziali**: il passaggio a una nuova fase avviene solo dopo il completamento della precedente.
- **Document-driven**: ogni fase produce documenti che devono essere approvati da un gruppo di valutatori (**Software Quality Assurance**) prima di procedere.
- **Focus sulla progettazione**: l'analisi e la progettazione sono centrali e ben documentate.



Vantaggi (Pro)

- Chiarezza nella **struttura del processo**.
- Documentazione completa, utile per progetti complessi e team numerosi.
- Riduce l'incertezza nella fase di sviluppo iniziale.

Svantaggi (Contro)

- **Rigidità del processo**: mancanza di interazione con il cliente, che vede il prodotto solo al termine dello sviluppo.
- **Rischio di discrepanze**: spesso il prodotto finale differisce da come il cliente lo immaginava.
- **Ripetizione del processo**: se emergono discrepanze, l'intero ciclo può dover essere rifatto.
- **Eccessiva produzione di documenti**: nessuna fase è considerata completa finché il documento corrispondente non è approvato, aumentando i tempi e i costi.

> Nota: il modello a cascata è utile per progetti con requisiti stabili e ben definiti, ma meno adatto a progetti che richiedono flessibilità e frequente interazione con il cliente.

Modello a V

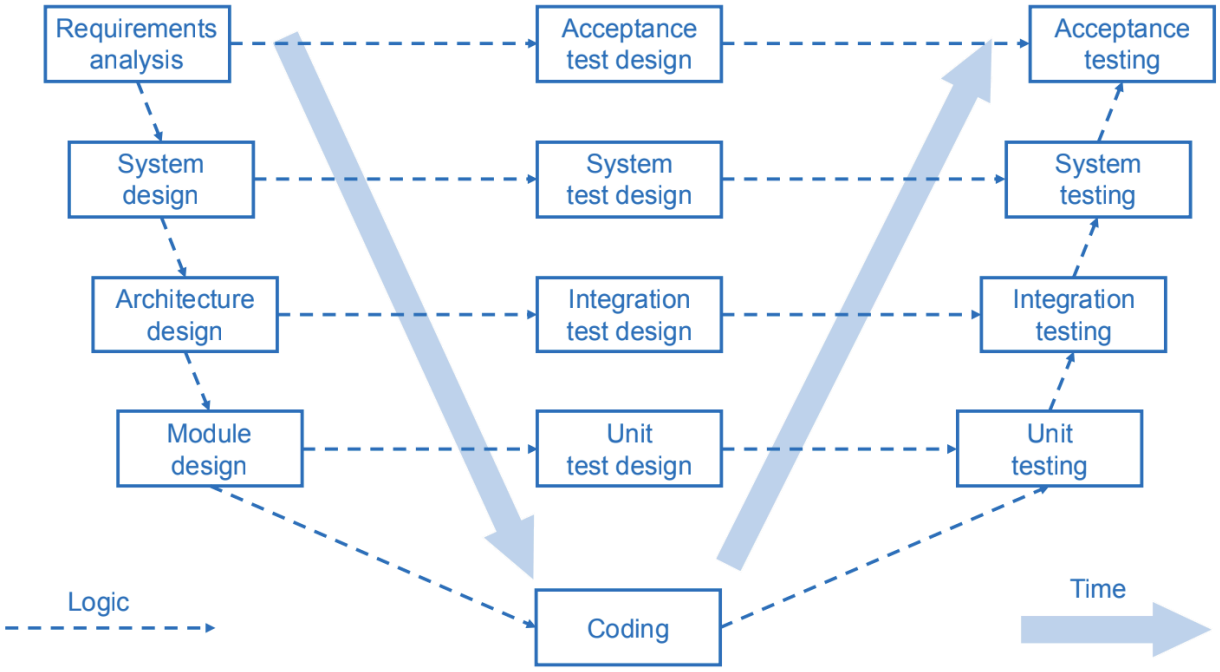
Il **modello a V** è un'evoluzione del modello a cascata: la metà inferiore della cascata viene **piegata verso l'alto a forma di V**, creando una struttura che enfatizza la **verifica e la convalida**.

Struttura del modello

- **Lato sinistro (SX)** – attività di analisi:
  - Scomposizione delle esigenze degli utenti in **parti piccole e gestibili**.
  - Focus sulla definizione chiara dei requisiti.
- **Lato destro (DX)** – attività di sintesi e test:

- Aggregazione delle parti sviluppate e **verifica che il sistema soddisfi i requisiti**.
- Include attività di test e validazione corrispondenti alle attività di analisi del lato SX.

- **Centro della V – progettazione dei test:**
  - I test vengono progettati **durante le fasi di sviluppo**, **prima della codifica**.
  - Evidenzia l'integrazione della progettazione del test come parte integrante del processo software.



Vantaggi principali

- Facilita la **verifica e convalida dei requisiti**.
- Evidenzia l'importanza della **progettazione dei test** fin dall'inizio.
- Standard riconosciuto nell'ambito della **Software Quality Assurance (SQA)**.
- Concetto alla base del **Test Driven Development (TDD)**: progettare i test prima di scrivere il codice.

> Nota: il modello a V mantiene la struttura sequenziale della cascata, ma migliora la gestione della qualità e riduce il rischio di discrepanze tra requisiti e prodotto finale.

Modelli sequenziali vs modelli iterativi

I **modelli di sviluppo software** possono essere classificati in base a come organizzano le fasi di lavoro: **sequenziali** o **iterativi**.

1. Modelli sequenziali

- Esempi: **Cascata**, **Modello a V**.
- **Caratteristiche principali**:
  - Le fasi si susseguono in ordine rigoroso.
  - Non si passa a una nuova fase finché quella precedente non è completata.
  - La progettazione e la definizione dei requisiti avvengono **all'inizio**.
- **Vantaggi**: chiarezza nella pianificazione, documentazione dettagliata.
- **Svantaggi**: poca flessibilità, difficile adattamento a cambiamenti dei requisiti o feedback del cliente.

2. Modelli iterativi

- Esempi: **Modello incrementale**, **Modello a spirale**, **Unified Process**.
- **Caratteristiche principali**:
  - Lo sviluppo avviene attraverso **ripetute iterazioni**, con rilascio di versioni parziali del sistema.
  - Ogni iterazione comprende fasi di analisi, progettazione, implementazione e test.
  - Permette di **integrare feedback continuo** da utenti o stakeholder.

- **Vantaggi**: flessibilità, riduzione del rischio, possibilità di correggere errori o modificare requisiti durante lo sviluppo.
- **Svantaggi**: richiede gestione più attenta delle versioni e del progetto, documentazione meno rigida.

> Sintesi:
>
> - I **modelli sequenziali** sono più adatti a progetti con requisiti stabili e ben definiti.
> - I **modelli iterativi** sono più indicati per progetti complessi, con requisiti in evoluzione o dove è importante ricevere feedback continuo dal cliente.

Rapid Prototyping

Il **Rapid Prototyping** è un approccio allo sviluppo software che prevede la **realizzazione rapida di un prototipo** del sistema per il cliente.

Obiettivi principali

- Permettere al committente di **sperimentare il sistema** in anticipo.
- Aiutare il cliente a **definire meglio i requisiti**.
- Basandosi sul feedback del prototipo, si procede alla **fase di specifica**.



Quando usarlo

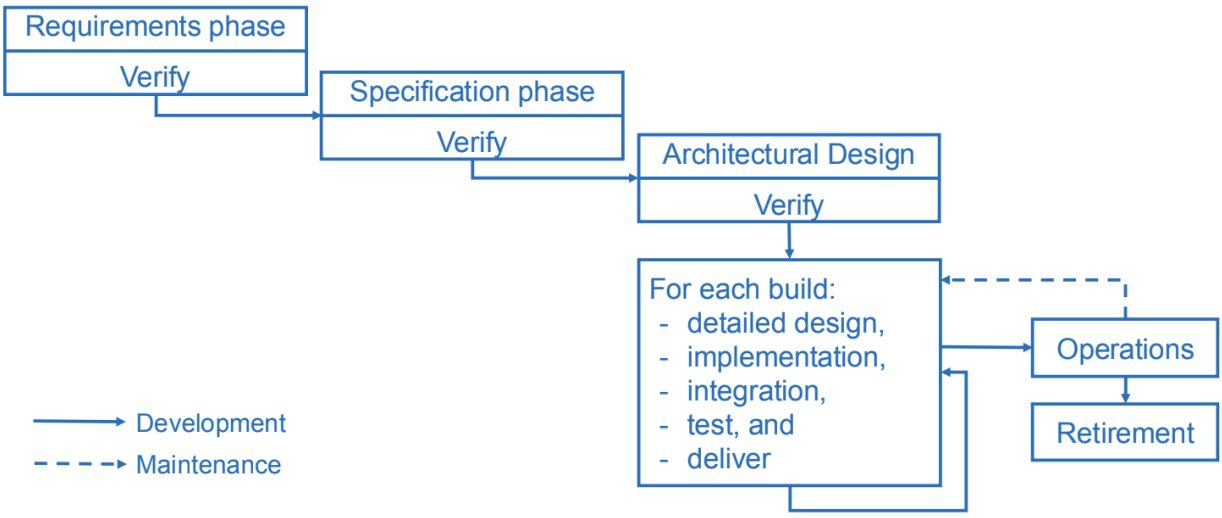
- Indicato quando i **requisiti iniziali non sono chiari** o sono difficili da definire completamente a priori.
- Utile per **ridurre incomprensioni** tra sviluppatori e committente.

Modello Incrementale

Il **modello incrementale** è un approccio iterativo allo sviluppo software in cui il sistema viene costruito **aggiungendo progressivamente nuove funzionalità**.

Caratteristiche principali

- I **requisiti del progetto** sono definiti inizialmente, almeno in parte.
- Il sistema viene **implementato, integrato e testato** attraverso **passaggi incrementali**, rilasciando versioni parziali e sempre più complete.



Vantaggi (Pro)

- Consente di **ritardare la realizzazione di componenti critiche** (es. tecnologie o hardware sperimentale).
- Permette di **rilasciare rapidamente prime versioni** del sistema, anche se con funzionalità ridotte.
- Il sistema è **utilizzabile in tempi brevi**.
- Facilita il **feedback continuo dai clienti**, migliorando l'allineamento tra esigenze e prodotto.
- Riduce il **costo dei cambiamenti** dei requisiti durante lo sviluppo.

Svantaggi (Contro)

- Il **processo software è meno visibile** rispetto a modelli sequenziali ben documentati.
- Se mal gestito, può degenerare in un approccio simile a **Build-and-Fix**.
- La **struttura del sistema tende a degradarsi** con l'aggiunta di nuovi incrementi, rendendo le modifiche future più difficili e costose.
- Richiede **tempo e risorse per il refactoring** continuo del codice.

Modello a Spirale (Bohem, 1988)

Il **modello a spirale** è un approccio iterativo allo sviluppo software che combina elementi dei modelli **incrementali** e di gestione del **rischio**. Ogni ciclo di sviluppo è rappresentato come **un giro della spirale**, con quattro fasi principali:

Fasi di ogni iterazione

1. **Definizione degli obiettivi**
   - Stabilire cosa deve essere realizzato in questo ciclo.
   - Identificare i requisiti funzionali e non funzionali da soddisfare.
2. **Analisi dei rischi**
   - Individuare potenziali problemi o ostacoli (tecnologici, economici, di progetto).
   - Valutare l'impatto dei rischi e definire strategie di mitigazione.
3. **Sviluppo e validazione**
   - Realizzare il software secondo gli obiettivi definiti.
   - Testare e validare i componenti sviluppati per verificare la correttezza e la qualità.
4. **Pianificazione del nuovo ciclo**
   - Valutare i risultati del ciclo corrente.
   - Pianificare il ciclo successivo della spirale con nuovi obiettivi, priorità e rischi da affrontare.



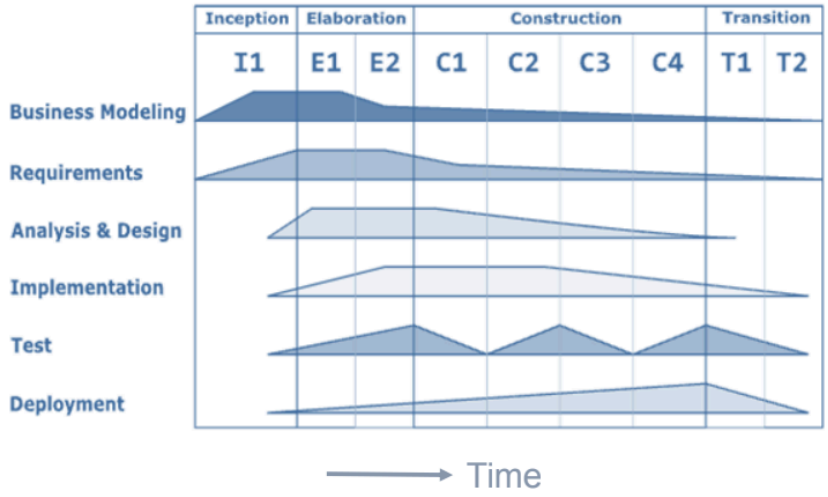
Caratteristiche principali

- **Modello astratto da istanziare**
  - Permette di definire **cosa fare in ogni iterazione e in ogni fase**.
  - Applicabile anche ai **cicli tradizionali** di sviluppo.
- **Aspetti gestionali evidenziati**
  - Pianificazione delle fasi di sviluppo.
  - **Risk-driven**: guidato dall'analisi dei rischi, ad esempio in presenza di domini poco conosciuti, linguaggi o strumenti nuovi, personale non addestrato.
  - Ispirato al **Plan-Do-Check-Act cycle** di William E. Deming (1950).
  - Promuove **maggiore comunicazione e confronto con il committente**, riducendo discrepanze tra prodotto atteso e prodotto realizzato.

> Nota: il modello a spirale è particolarmente utile per progetti complessi e con requisiti incerti, perché integra lo sviluppo iterativo con una gestione costante dei rischi e un forte coinvolgimento del cliente.

Unified Process (Booch et al., 1999)

Il **Unified Process (UP)** è un modello di sviluppo **iterativo e incrementale**, guidato da **casi d'uso** e dall'**analisi dei rischi**.



Caratteristiche principali

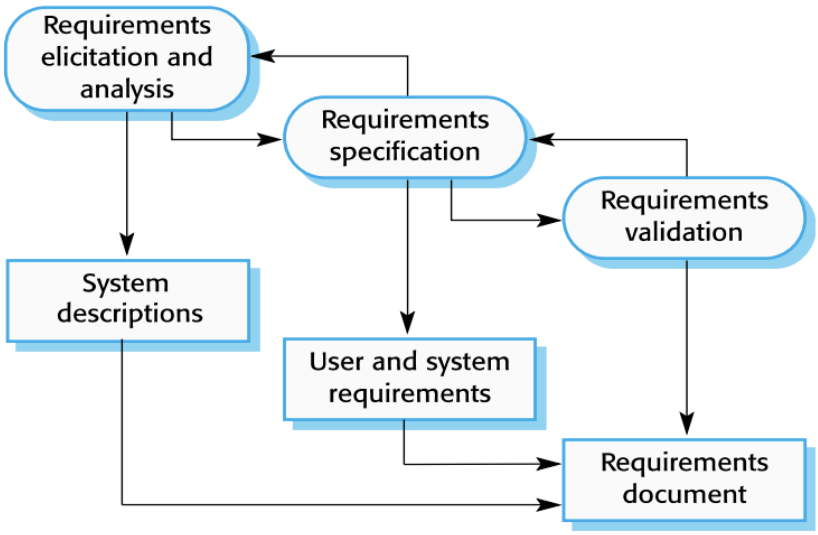
- **Guidato da casi d'uso e rischi**
  - Fin dalle prime fasi di raccolta e analisi dei requisiti.
  - Permette di identificare funzionalità critiche e aree a rischio del progetto.
- **Modello iterativo e incrementale incentrato sull'architettura**
  - Nelle prime fasi viene definita un'**architettura di massima** del sistema.
  - I dettagli vengono sviluppati nelle iterazioni successive.
  - Offre subito una **visione generale del sistema**, facilitando l'adattamento ai cambiamenti dei requisiti.

> Nota: grazie all'approccio iterativo e alla centralità dell'architettura, il Unified Process combina flessibilità e controllo, riducendo i rischi di discrepanze tra requisiti e prodotto finale.

Fasi del Processo Software

1. Fase di Specifica

- **Obiettivo:** Stabilire quali servizi sono richiesti e i vincoli sul funzionamento del sistema.
- **Processo di ingegneria dei requisiti:**
  1. **Estrazione e analisi dei requisiti:** cosa richiedono o si aspettano gli stakeholder dal sistema.
  2. **Specifica dei requisiti:** definizione dettagliata dei requisiti.
  3. **Convalida dei requisiti:** verifica della correttezza e completezza dei requisiti.



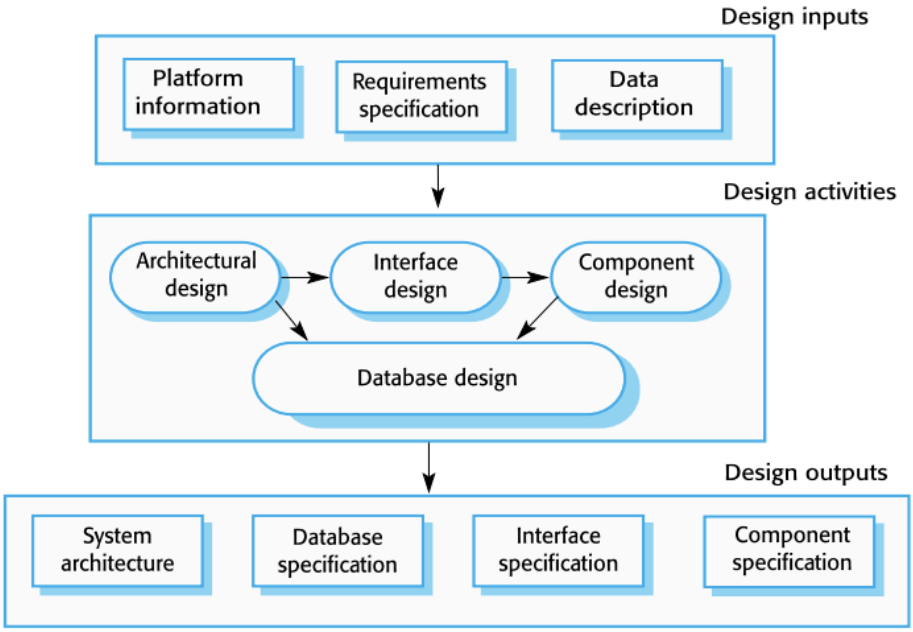
Da: I. Sommerville. **Ingegneria del Software** (10ed), Pearson, 2017

2. Fase di Progettazione

- **Obiettivo:** Definire una struttura software che realizzi la specifica.
- **Tipi di progettazione:**
  - **Architectural design:** identificazione della struttura del sistema (componenti, relazioni, distribuzione).
  - **Database design:** definizione delle strutture dati e della loro rappresentazione.

- **Interface design:** definizione delle interfacce tra componenti.
- **Component design:** definizione dettagliata dei componenti, incluso eventuale riuso di componenti esistenti.

- **Nota:** Inizia la conversione della specifica in un sistema eseguibile.



Da: I. Sommerville. **Ingegneria del Software** (10ed), Pearson, 2017

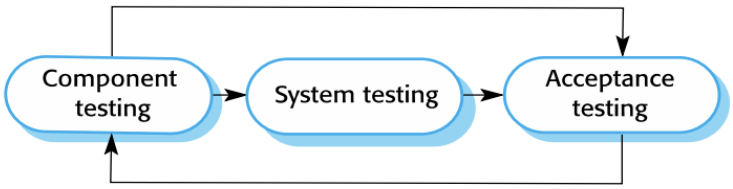
3. Fase di Sviluppo

- **Obiettivo:** Realizzare il software implementando programmi o configurando sistemi esistenti.
- **Attività principali:**
  - **Programmazione:** sviluppo del codice (individuale o collaborativo).
  - **Debugging:** identificazione e correzione degli errori.
- **Nota:** progettazione e sviluppo sono spesso **interleaved** (eseguite in parallelo).

---

4. Fase di Validazione

- **Obiettivo:** Verificare che il sistema sia conforme alle specifiche e soddisfi i requisiti del cliente.
- **Attività principali:**
  - **Component testing:** test dei singoli componenti in modo indipendente.
  - **System testing:** test del sistema nella sua interezza, con attenzione alle proprietà emergenti.
  - **Customer testing:** test con dati reali forniti dal cliente.
- **Metodo:** esecuzione di casi di test derivati dai requisiti e dai dati reali.



Da: I. Sommerville. **Ingegneria del Software** (10ed), Pearson, 2017

5. Fase di Evoluzione

- **Motivazioni:** I cambiamenti sono inevitabili: nuovi requisiti, nuove tecnologie, miglioramenti.
- **Attività di rework:**
  - Analisi e aggiornamento dei requisiti.
  - Progettazione e sviluppo di nuove funzionalità.
  - Adattamento di funzionalità esistenti.
- **Gestione dei cambiamenti:**
  - **Change tolerance:** organizzare il processo per implementare cambiamenti a costi contenuti.
  - Modelli **incrementali** facilitano l'aggiornamento, consentendo modifiche solo sui nuovi incrementi o sulle parti interessate.

Evoluzione rapida e modelli agili

- **Contesto:** i requisiti cambiano velocemente (fast-changing business requirements), rendendo impossibile stabilire un insieme stabile di requisiti.
- **Soluzione:** modelli agili (fine anni ’90) per ridurre i tempi di rilascio:
  - Specifica, progettazione e sviluppo sono **interleaved**.
  - Sviluppo come serie di **versioni incrementali**, con stakeholder coinvolti nella definizione e valutazione.
  - **Rilascio frequente** per test e feedback delle nuove funzionalità.
  - Supporto da strumenti di sviluppo moderni (es. **automated testing**).

Plan-Driven vs Agile

I processi software possono essere classificati in base all’approccio alla gestione e allo sviluppo: **plan-driven** (basati sulla pianificazione) e **agile** (flessibili e iterativi).

1. Plan-Driven

- **Caratteristiche principali:**
  - Pianificazione dettagliata **prima dell’inizio del progetto**.
  - Ogni fase (specifica, progettazione, sviluppo, test) è **predeterminata**.
  - Forte enfasi sulla **documentazione**.
  - Modelli tipici: **Cascata**, **Modello a V**, **Modello a Spirale** (parzialmente).
- **Vantaggi:**
  - Struttura chiara e prevedibile.
  - Controllo rigoroso di tempi, costi e qualità.
- **Svantaggi:**
  - Poco flessibile ai cambiamenti dei requisiti.
  - Rischio di discrepanze tra prodotto finale e aspettative del cliente.

2. Agile

- **Caratteristiche principali:**
  - Ciclo iterativo e incrementale.
  - **Collaborazione continua** con gli stakeholder.
  - Minima documentazione; focus su **software funzionante**.
  - Le fasi di specifica, progettazione e sviluppo sono **interleaved**.
  - Rilascio frequente di versioni incrementali per ricevere **feedback continuo**.
  - Modelli tipici: **Scrum**, **Extreme Programming (XP)**, **Continuous Delivery**.
- **Vantaggi:**
  - Alta flessibilità ai cambiamenti dei requisiti.
  - Feedback rapido dal cliente e miglioramento continuo.
  - Riduzione dei tempi di rilascio.
- **Svantaggi:**
  - Meno prevedibile in termini di costi e tempi.
  - Richiede forte disciplina del team e strumenti di supporto.

> Sintesi:
>
> - **Plan-driven** è adatto a progetti con requisiti stabili e team grandi o distribuiti.
> - **Agile** è ideale per progetti complessi, con requisiti in evoluzione e necessità di **rilasci rapidi e frequenti**.